

Bodybuilding Coaching Platform

Optimized Technical & Product Specification — Prepared for Gnaneswar Kokkiralala

This document restructures your original prompt into a build-ready spec: a phased MVP-first roadmap, a trimmed and de-risked tech stack, a full page-by-page feature breakdown, API reference, project folder structure, and a week-by-week build order — so you can start coding immediately instead of guessing what to build first.

1. What This Optimization Changes

- **Phased delivery** instead of one giant build — MVP first (bookable, payable, usable), then AI features, then full CMS.
- **Tech stack de-risked**: swapped bleeding-edge/unreleased versions for stable, production-proven equivalents.
- **Scope split** into Must-Have (MVP), Should-Have (V2), and Nice-to-Have (V3).
- **Payment flow tightened** into a concrete verification state machine.
- **Security and auth hardened** with explicit rules.
- **New in this version**: page-by-page feature spec, full API endpoint list, project folder structure, environment variables, and a week-by-week build order.

2. Recommended Tech Stack

Your original stack named Next.js 16 / React 19 as ‘2026’ versions — those are either unreleased or brand-new at time of writing, which is a real project risk (broken packages, no fixes, no Shadcn/Framer compatibility yet). Pin to the latest **stable** versions at build time; treat the table below as a floor, not a fixed target.

Layer	Recommended	Why
Frontend framework	Next.js (latest stable, App Router)	SSR/SEO for marketing pages, fast dashboards
UI	React + TypeScript + Tailwind CSS + Shadcn/UI	Solid, well-supported combo, matches original spec
Motion	Framer Motion (UI state) + GSAP (hero/scroll only)	Don't use both everywhere — GSAP for scroll-heavy sections
Backend	FastAPI (Python 3.12+ stable)	Async, fast, integrates cleanly with AI endpoints
ORM / Validation	SQLAlchemy 2.x + Pydantic v2	As specified
Database	SQLite (MVP) → PostgreSQL (V2 launch)	SQLite fine for dev/low traffic; Postgres before real scale
Auth	JWT (access + refresh) + bcrypt	Add refresh-token rotation and role claims
File storage	Local disk (MVP) → S3-compatible (V2)	Move screenshots/media off local disk before production traffic
AI Planner	Groq API (LLaMA 3.3) via server-side FastAPI routes	Never call the AI provider from the browser

3. Phased Roadmap

Phase 1 — MVP (Launch-ready core)

Goal: a real, usable coaching site that can take clients and payments.

- Pages: Home, About, Programs, Pricing, Transformations, Contact, FAQ, Login, Register, User Dashboard, Admin Dashboard (basic).
- Manual UPI/QR payment with screenshot upload + admin approve/reject flow.
- JWT auth with role-based access (user / admin).
- Admin can upload/edit: pricing, programs, testimonials, transformation gallery — via a simple form-based CMS.
- User dashboard: view assigned plan, download PDF, view payment status.

Phase 2 — Growth Features

- AI Workout Planner and AI Diet Planner (Groq/LLaMA 3.3), with editable AI output before saving to a user's plan.
- Exercise Library, Nutrition section, Blog, Community.
- Progress Tracker with charts (weight, measurements, photos).
- Migrate storage: SQLite → PostgreSQL, local files → object storage.

Phase 3 — Commercial-Grade Polish

- Full drag-and-drop visual page editor for the Super Admin CMS.
- Shop module (products, cart, orders).
- Analytics dashboard (traffic, conversion, revenue).
- Swap manual UPI for Razorpay/Stripe automated payments.
- WhatsApp Business API integration.

4. Page-by-Page Feature Spec

What each page actually needs to contain and do — use this as your component checklist.

Page	Must contain	Phase
Home	Hero w/ coach photo, results stats (4+ yrs, 10+ transformations), CTA to Pricing, testimonial strip, featured transformation	
About	Coach bio, certifications, philosophy, contact CTA	1
Programs	Program cards (beginner/intermediate/advanced), what's included, price, 'Buy Now'	1
Pricing	Plan comparison table, UPI/QR payment, screenshot upload form	1
Transformations	Before/after gallery, filter by goal (fat loss/muscle gain), client story text	1
Contact	Phone (tap-to-call), WhatsApp link, email form, embedded map, social icons	1
FAQ	Accordion list, searchable if list grows past ~15 items	1
Login / Register	Email+password, JWT issue, form validation, 'forgot password' stub for V2	1
User Dashboard	Active plan card, download plan PDF, payment status, progress summary	1
Admin Dashboard	Pending payments queue, approve/reject action, basic content edit forms	1
Workout Plans	Plan list, filter by goal/level, PDF download, AI-generate button (Phase 2)	1/2
Diet Plans	Same pattern as Workout Plans, macros summary per plan	1/2
AI Workout/Diet Planner	Form (goal, level, equipment, diet type) → Groq call → editable result → save to dashboard	2
Exercise Library	Searchable/filterable exercise cards, muscle group tags, short demo clip or image	2
Nutrition	Articles/guides, macro calculator widget	2
Progress Tracker	Weight/measurement log form, line chart, photo upload timeline	2
Blog	Post list, category filter, single post SSR page for SEO	2
Community	Discussion feed or simple comment wall (scope this down for V2 — full forums are a V3+ feature)	2
Shop	Product grid, cart, checkout tied into payment flow	3
Super Admin CMS	Full visual editor for all sections above, media library, analytics view	3

5. Payment Verification Flow

The original prompt said 'admin verifies' without defining states. Use this flow so both the code and the admin UI share one source of truth:

- **pending** — user submits order, uploads UPI/QR payment screenshot.
- **under_review** — admin sees it in a review queue.
- **approved** — admin confirms; user's plan/purchase unlocks automatically; confirmation sent.
- **rejected** — admin rejects with a reason; user is notified and can re-upload.
- Log every state transition with admin ID + timestamp for dispute resolution.

6. Security & Auth Requirements

- Access tokens short-lived (~15 min); refresh tokens rotated and stored httpOnly.
- Passwords hashed with bcrypt (cost factor 12+); never log raw passwords or tokens.
- Role-based route guards on both frontend (UI hiding) and backend (real authorization).
- Rate-limit login, register, and file-upload endpoints.
- Validate/sanitize all uploads: type, size, re-encode images before storing.
- Admin actions (pricing edits, plan edits, payment approvals) logged in an audit trail.

7. Core API Endpoint Reference

Suggested FastAPI route groups for the MVP — expand per phase as features are added.

Endpoint	Method	Purpose
/api/auth/register	POST	Create user account
/api/auth/login	POST	Issue access + refresh JWT
/api/auth/refresh	POST	Rotate access token
/api/users/me	GET	Current user profile
/api/programs	GET	List programs/plans
/api/programs/{id}	GET	Program detail
/api/orders	POST	Create order + upload payment screenshot
/api/orders/me	GET	User's own order/payment history
/api/admin/orders	GET	Admin: list pending/under_review orders
/api/admin/orders/{id}/approve	POST	Admin: approve payment, unlock plan
/api/admin/orders/{id}/reject	POST	Admin: reject payment with reason
/api/plans/me	GET	User's unlocked workout/diet plans
/api/ai/workout-plan	POST	Server-side Groq call → generated plan (Phase 2)
/api/ai/diet-plan	POST	Server-side Groq call → generated plan (Phase 2)
/api/progress	POST / GET	Log and fetch progress entries (Phase 2)
/api/admin/content/{section}	GET / PUT	CMS: edit site sections (pricing, testimonials, gallery)

8. Suggested Project Folder Structure

```
bodybuilding-platform/
├── frontend/                                (Next.js app)
│   ├── app/
│   │   ├── (public)/                       home, about, programs, pricing, blog...
│   │   ├── (auth)/                         login, register
│   │   ├── dashboard/                     user dashboard routes
│   │   ├── admin/                         admin dashboard routes
│   │   ├── components/                    shared UI (Shadcn-based)
│   │   ├── lib/                           api client, auth helpers
│   │   └── styles/
│   └──
├── backend/                                (FastAPI app)
│   ├── app/
│   │   ├── api/                           route modules (auth, orders, programs, ai...)
│   │   ├── models/                        SQLAlchemy models
│   │   ├── schemas/                       Pydantic schemas
│   │   ├── services/                      business logic (payment verification, AI calls)
│   │   ├── core/                          config, security (JWT, bcrypt)
│   │   └── main.py
│   ├── uploads/                           local file storage (screenshots, media) – MVP only
│   ├── requirements.txt
│   └──
└── docs/                                  this spec, API docs, changelog
```

9. Environment Variables (starting set)

```
# backend/.env
DATABASE_URL=sqlite:///./app.db           # swap for postgres:// in Phase 2
JWT_SECRET=change_me
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
GROQ_API_KEY=your_groq_key                # server-side only, never exposed to frontend
UPLOAD_DIR=./uploads
ADMIN_UPI_ID=coach@upi

# frontend/.env.local
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
```

10. Week-by-Week Build Order (MVP)

Week	Focus
1	Backend skeleton: FastAPI project, SQLAlchemy models (User, Program, Order), JWT auth (register/login/refresh).
2	Frontend skeleton: Next.js + Tailwind + Shadcn setup, layout/nav, Home/About/Programs/Pricing pages (static content).
3	Payment flow: order creation, screenshot upload, admin review queue, approve/reject endpoints + UI.
4	User dashboard (plan view, payment status) + Admin dashboard (basic content edit forms for pricing/testimonials/gallery).
5	Contact, FAQ, Transformations gallery, polish responsive/mobile layout, deploy MVP.

11. Non-Functional Requirements

- **Performance:** Lighthouse 90+ on Home/Programs/Pricing; lazy-load gallery and blog images.
- **SEO:** SSR/static generation for public pages (Home, About, Blog, Programs).
- **Accessibility:** WCAG AA contrast even within the dark luxury theme; keyboard-navigable forms.
- **Mobile-first:** most coaching-site traffic is mobile — design and test mobile before desktop.

12. Core Data Model

Entity	Key Fields
User	id, name, email, phone, password_hash, role, created_at
Program / Plan	id, title, description, price, type (workout/diet), pdf_url
Order / Payment	id, user_id, plan_id, amount, screenshot_url, status, reviewed_by, reviewed_at
Transformation	id, user_id (optional), before_img, after_img, story, is_public
Progress Entry	id, user_id, date, weight, measurements(json), photo_url
Blog Post	id, title, slug, body, cover_img, published_at
Media Asset	id, file_path, type, uploaded_by, uploaded_at

13. Bottom Line

Build Phase 1 first and get it live — a real coaching business running on a solid MVP beats a perfect spec that never ships. Everything in Phase 2 and 3 slots into the same architecture without a rewrite, as long as the data model, API structure, and auth/role rules above are followed from day one.